

1. Rola modułu w systemie Stop & Go

Moduł `driver.py` jest ostatnim ogniwem łańcucha — interfejsem między oprogramowaniem a sprzętem. Przyjmuje wartość PWM [0.0, 1.0] z `controller.py` i zamienia ją na fizyczne sygnały elektryczne na pinach GPIO Raspberry Pi, które sterują sterownikiem silników L298N, który z kolei zasila silnik DC napędzający pojazd.

Moduł jest zaprojektowany z myślą o dwóch trybach pracy: trybie symulacji (`simulate=True`, domyślny) który działa na każdym komputerze bez sprzętu, oraz trybie sprzętowym (`simulate=False`) który wymaga Raspberry Pi z podłączonym L298N. Dzięki temu cały projekt można rozwijać i testować na laptopie, a przełączenie na sprzęt to zmiana jednej zmiennej w `main.py`.

2. Silnik DC — zasada działania

Silnik prądu stałego (DC) zamienia energię elektryczną na mechaniczną przez wzajemne oddziaływanie pola magnetycznego stałego (magnes trwały w obudowie) i pola magnetycznego zmiennego (uzwojenia wirnika zasilane prądem). Prąd przepływający przez uzwojenia wirnika wytwarza moment magnetyczny który jest odpychany lub przyciągany przez magnesy stałe — powstaje moment obrotowy.

Dwie fundamentalne właściwości silnika DC istotne dla projektu: kierunek obrotu zależy od kierunku prądu przez uzwojenia (odwrócenie prądu = odwrócenie kierunku), a prędkość obrotu jest proporcjonalna do napięcia zasilania. Obie te właściwości są sterowane przez mostek H i sygnał PWM.

Prąd rozruchowy a Raspberry Pi:

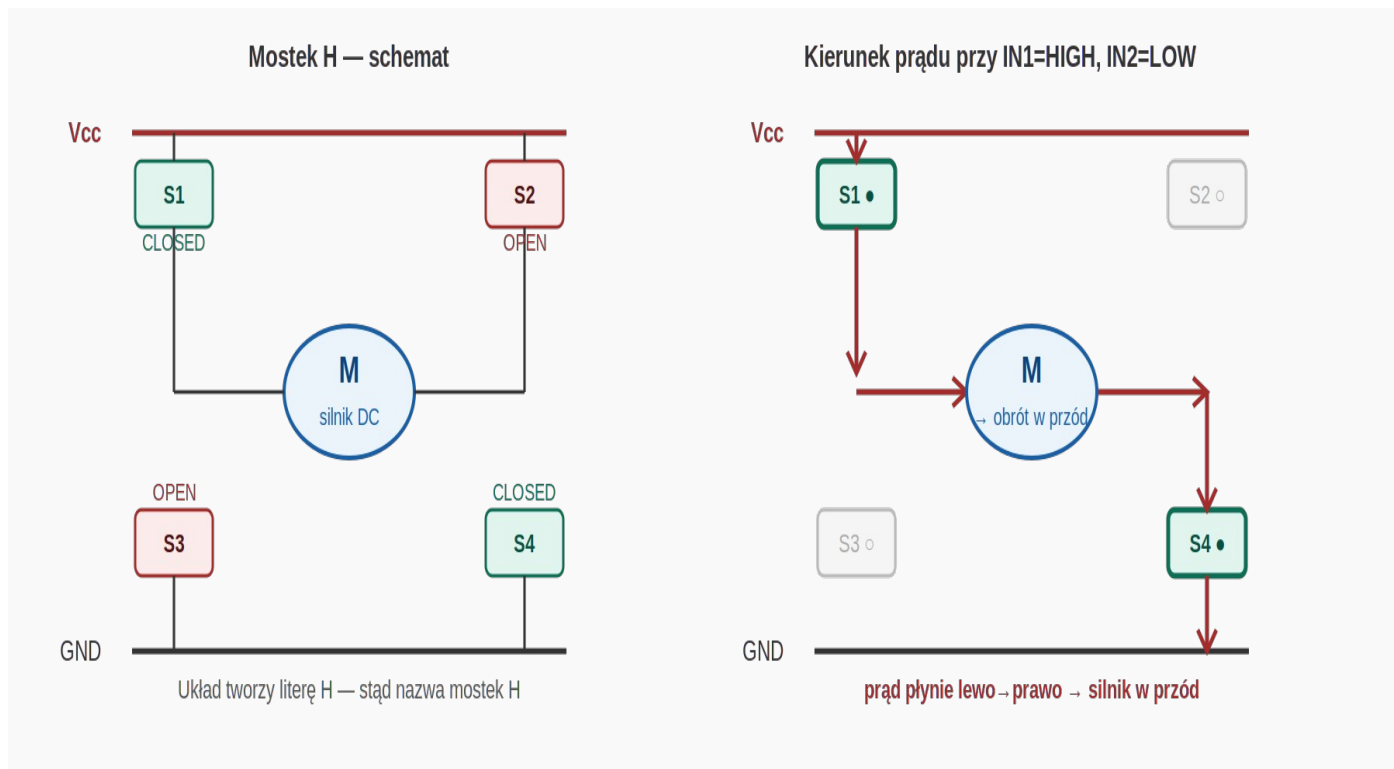
W chwili startu ($n=0$ obr/min) silnik DC pobiera prąd rozruchowy wielokrotnie wyższy niż nominalny. Dla silnika 1A nominalny przy $U=6V$ i $R=1\Omega$ uzwojeń: $I_{rozr} = U/R = 6/1 = 6A$.

GPIO Raspberry Pi może dostarczyć maksymalnie 16 mA na pin i ~50 mA łącznie. Dlatego nigdy nie podłączamy silnika bezpośrednio do GPIO — zawsze przez sterownik L298N który ma własne zasilanie dla silników.

RPi steruje tylko sygnałami logicznymi (3.3V, mikroampery). L298N dostarcza silnikowi pełen prąd z zewnętrznego zasilacza.

3. Mostek H — sterowanie kierunkiem silnika DC

Mostek H (H-bridge) to układ czterech przełączników (tranzystorów) ułożonych symetrycznie po obu stronach silnika. Nazwa pochodzi od kształtu schematu elektrycznego który przypomina literę H — dwa pionowe słupki połączone poprzeczką (silnikiem). Jest to standardowe rozwiązanie do sterowania silnikami DC w robotyce i przemyśle.



Rysunek 1. Schemat mostka H (lewy panel) i kierunek prądu przy $IN1=HIGH$, $IN2=LOW$ (prawy panel). Zamknięcie $S1+S4$ powoduje obrót w przód, $S2+S3$ — w tył.

3.1 Tabela prawdy mostka H

$IN1=HIGH$, $IN2=LOW$	$S1$ i $S4$ zamknięte → prąd płynie lewo→prawo przez silnik → obrót w PRZÓD
$IN1=LOW$, $IN2=HIGH$	$S2$ i $S3$ zamknięte → prąd płynie prawo→lewo przez silnik → obrót w TYŁ
$IN1=LOW$, $IN2=LOW$	Wszystkie przełączniki otwarte → silnik swobodnie się obraca (coast mode)
$IN1=HIGH$, $IN2=HIGH$	ZABRONIONE — zwarcie przez mostek H, uszkodzenie L298N lub zasilacza

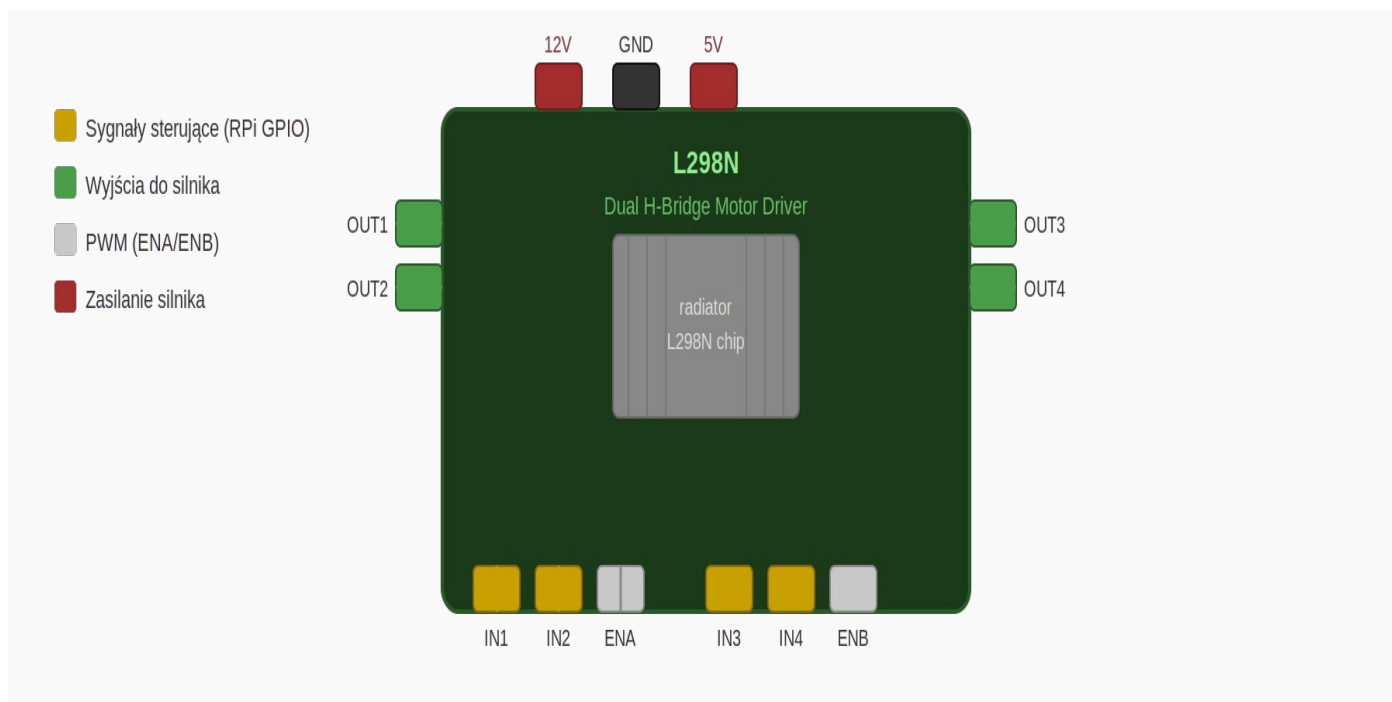
Dlaczego $IN1$ i $IN2$ zawsze muszą mieć przeciwne wartości?

W kodzie: `self._GPIO.output(self._pin_a, HIGH if forward else LOW)`
`self._GPIO.output(self._pin_b, LOW if forward else HIGH)`

Te dwie linie gwarantują że `pin_a` i `pin_b` są zawsze przeciwne.
 Nigdy oba HIGH jednocześnie — to chroni L298N przed zwarcieniem.
 Jest to hardwarowe wymaganie bezpieczeństwa mostka H.

4. Sterownik L298N — budowa i interfejs

L298N (ST Microelectronics) to scalony układ zawierający dwa niezależne mostki H zdolne do sterowania dwoma silnikami DC lub jednym silnikiem krokowym. Maksymalny prąd wyjściowy to 2A na kanał (4A szczytowo) przy napięciu zasilania silnika 5–46V. Jest to jedno z najpopularniejszych rozwiązań w robotyce amatorskiej i edukacyjnej.



Rysunek 2. Schemat modułu L298N z opisem pinów. ENA steruje prędkością przez PWM, IN1/IN2 sterują kierunkiem, OUT1/OUT2 to wyjścia do silnika.

4.1 Opis pinów L298N używanych w projekcie

ENA	Enable A — wejście PWM dla kanału A. Wypełnienie 0–100% = prędkość 0–100%. Podłączone do BCM12 (sprzętowy PWM RPi).
IN1 (pin_dir_a)	Wejście kierunku A, bit 1. Podłączone do BCM23. Logika 3.3V RPi jest tolerowana przez L298N.
IN2 (pin_dir_b)	Wejście kierunku A, bit 2. Podłączone do BCM24. Zawsze przeciwne do IN1.
OUT1, OUT2	Wyjścia do silnika DC. Napięcie i prąd z zewnętrznego zasilacza (nie z RPi).
VCC (12V)	Zasilanie silnika — osobne od zasilania RPi. Typowo 6–12V dla silników DC modelu RC.
GND	Masa wspólna dla RPi i L298N. Musi być połączona — bez wspólnej masy sygnały logiczne nie działają.

Wspólna masa (GND) — krytyczne połączenie:

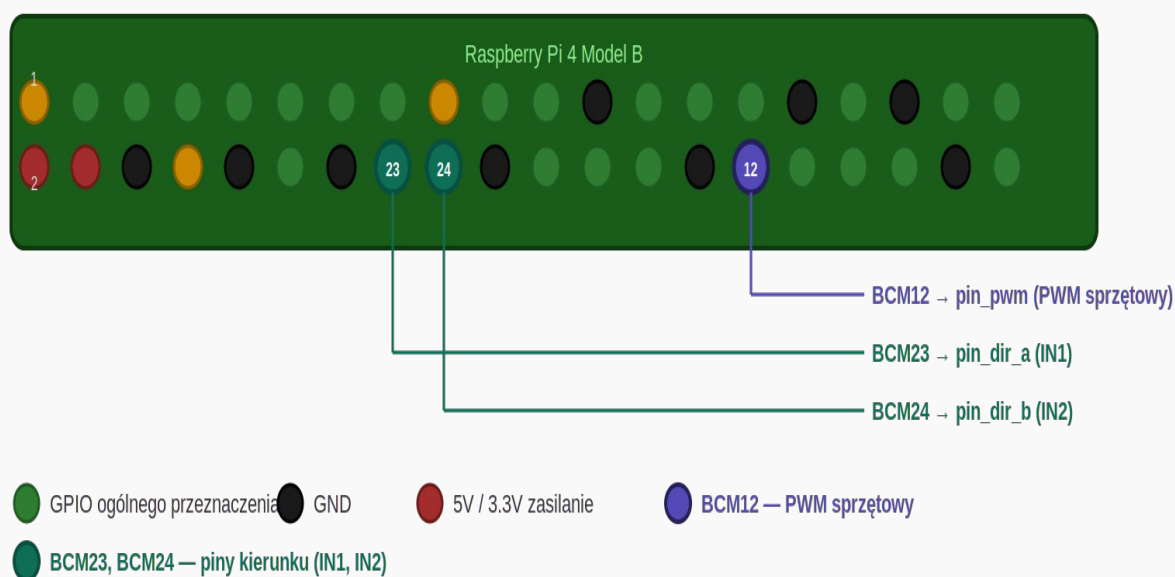
RPi i L298N muszą mieć połączone GND nawet jeśli mają osobne zasilacze. Bez wspólnej masy sygnał logiczny HIGH (3.3V z RPi) nie ma odniesienia do napięcia L298N i może być interpretowany jako LOW lub jako szum.

Typowy błąd przy pierwszym uruchomieniu: silnik nie reaguje na sygnały pomimo poprawnych obliczeń PWM → sprawdzić czy GND jest połączony.

5. GPIO Raspberry Pi — interfejs sprzętowy

Raspberry Pi 4 Model B posiada 40-pinowe złącze GPIO (General Purpose Input/Output). Piny mogą pracować jako wejścia cyfrowe, wyjścia cyfrowe, lub jako interfejsy specjalnych funkcji (PWM, SPI, I2C, UART). Maksymalne napięcie logiczne to 3.3V — bezwzględnie nie podłączamy napięcia 5V do pinów GPIO.

Raspberry Pi 4 — złącze GPIO 40-pin (schemat)



Rysunek 3. Schemat złącza GPIO Raspberry Pi 4 z zaznaczonymi pinami używanymi w projekcie. BCM12 (fioletowy) to sprzętowy PWM, BCM23 i BCM24 (zielone) to piny kierunku.

5.1 BCM vs BOARD — dwie konwencje numerowania

BCM vs BOARD — dwie konwencje numerowania pinów GPIO



Ważne: GPIO.setmode() musi być wywołane PRZED GPIO.setup() — ustawia globalną konwencję dla całego programu

Rysunek 4. Porównanie konwencji BCM i BOARD. Projekt używa BCM bo jest przenośna między modelami RPi i jednoznacznie identyfikuje funkcję pinu.

5.2 Sprzętowy PWM — BCM12

Pin BCM12 (fizyczny pin 32) jest jednym z czterech pinów Raspberry Pi obsługujących sprzętowy PWM. Sprzętowy PWM jest generowany przez dedykowany timer w chipie BCM2837 — niezależnie od obciążenia procesora, sygnał PWM ma stałą i precyzyjną częstotliwość.

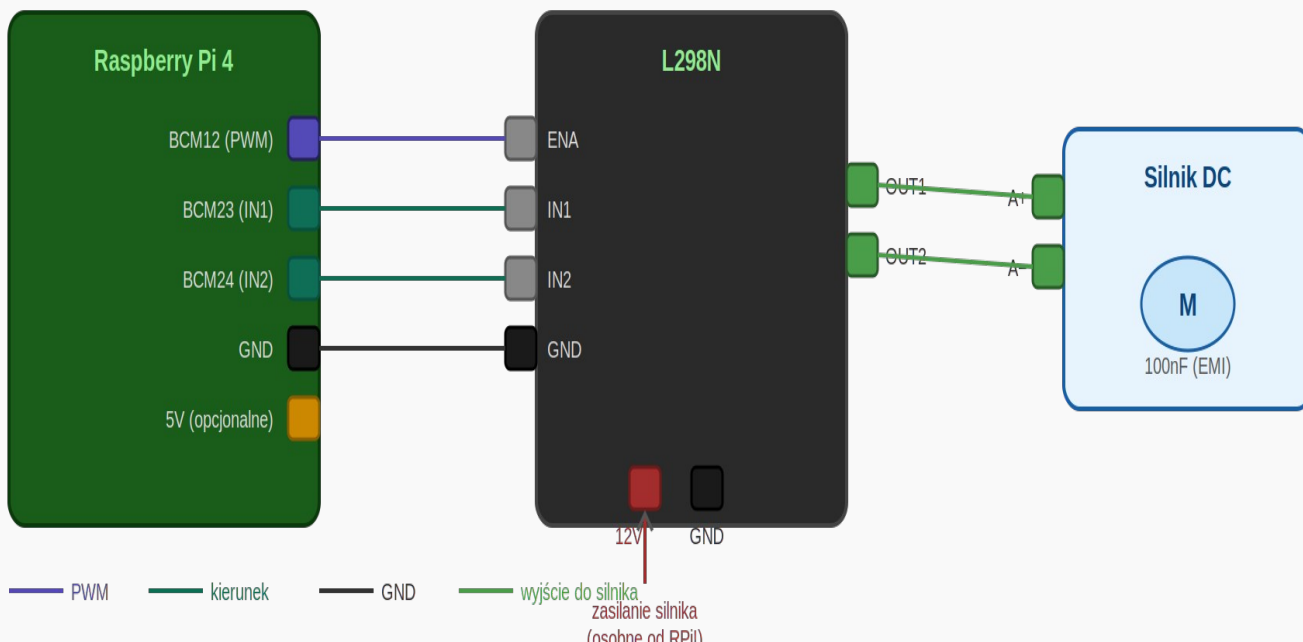
W odróżnieniu od softwarowego PWM (generowanego przez RPi.GPIO.PWM w Pythonie), sprzętowy PWM nie traci dokładności gdy CPU jest zajęty detekcją HOG lub śledzeniem obiektu. Dla sterowania silnikiem DC jest to istotna zaleta — niestabilne PWM powoduje drgania prędkości i niejednorodne zachowanie.

```
# Sprzętowy PWM na BCM12 — precyzyjny, niezależny od CPU
self._pwm_pin = GPIO.PWM(12, 1000) # pin 12, częstotliwość 1 kHz
self._pwm_pin.start(0) # start z 0% wypełnienia
self._pwm_pin.ChangeDutyCycle(55) # 55% = PWM 0.55
```

```
# Softwarowy PWM na GPIO17 — mniej dokładny przy obciążeniu CPU
# Unika się w systemach czasu rzeczywistego
```

6. Schemat połączeń RPi ↔ L298N ↔ Silnik

Schemat połączeń: Raspberry Pi ↔ L298N ↔ Silnik DC



Rysunek 5. Kompletny schemat połączeń. Kluczowe: zasilanie silnika (12V) jest osobne od zasilania RPi (5V USB). Wspólna masa (GND) łączy oba układy.

6.1 Dlaczego osobne zasilanie dla silnika?

Silnik DC podczas pracy pobiera prąd impulsowo — zwłaszcza przy ruszaniu i gwałtownych zmianach prędkości. Te impulsy prądowe powodują spadki napięcia na szynie zasilającej. Zasilacz USB Raspberry Pi (typowo 3A) może nie być w stanie dostarczyć jednocześnie wystarczającego prądu dla RPi (ok. 1.5A przy pełnym obciążeniu CPU) i dla silnika (1–3A).

Osobne zasilanie silnika przez L298N eliminuje ten problem. RPi pobiera prąd ze swojego zasilacza USB, silnik pobiera prąd z baterii lub zasilacza 12V przez L298N. Jedyne połączenie elektryczne między układami to przewody sygnałowe (BCM12, BCM23, BCM24) i wspólna masa.

Kondensatory odsprężające — ochrona RPi przed EMI silnika:

Silnik DC generuje podczas pracy zakłócenia elektromagnetyczne (EMI) przez szczotki i komutator. Te impulsy napięciowe mogą cofać się po przewodach zasilających i resetować Raspberry Pi.

Rozwiązanie: kondensatory ceramiczne 100 nF między każdym zaciskiem silnika a masą (+ jeden między biegunami). Montaż bezpośrednio przy zaciskach silnika — im bliżej, tym skuteczniejsze tłumienie EMI.

Ferryty na przewodach sygnałowych IN1/IN2/ENA dodatkowo tłumią zakłócenia wysokoczęstotliwościowe przedostające się do GPIO.

7. Tryb symulacji i klasa `_GPIOMock`

Warunkowy import `RPi.GPIO` i klasa `_GPIOMock` umożliwiają uruchamianie całego projektu na komputerze bez Raspberry Pi. Jest to kluczowa możliwość w procesie rozwoju — algorytmy detekcji, maszyna stanów i kontroler mogą być testowane i debugowane na laptopie zanim w ogóle pojawi się sprzęt.

```
class _GPIOMock:
    BCM = 'BCM'
    OUT = 'OUT'
    HIGH = True
    LOW = False

    def setmode(self, mode):
        pass
    def setup(self, pin, mode):
        pass
    def output(self, pin, value):
        pass
    def cleanup(self):
        pass

    class PWM:
        def __init__(self, pin, freq):
            pass
        def start(self, duty):
            pass
        def stop(self):
            pass
        def ChangeDutyCycle(self, duty):
            pass
```

`_GPIOMock` implementuje dokładnie ten sam interfejs co `RPi.GPIO` — te same metody, te same stałe. Dzięki temu kod `set_speed()` wykonuje się identycznie w obu trybach, różnica jest tylko w efekcie: prawdziwe GPIO zmienia stan pinu, mock nie robi nic. To jest wzorzec Test Double (Test Stub) z metodologii TDD (Test-Driven Development).

8. Interfejs publiczny

<code>set_speed(pwm)</code>	Główna metoda. Przyjmuje float [-1.0, 1.0]. Wartości dodatnie = jazda w przód, ujemne = wstecz. Wywołać z wartością z <code>controller.py</code> .
<code>stop()</code>	Natychmiastowe zatrzymanie. Deleguje do <code>set_speed(0.0)</code> . Działa w obu trybach.
<code>cleanup()</code>	Zwalnia zasoby GPIO. Wywołać przy zamykaniu programu. Bez <code>cleanup()</code> piny pozostają skonfigurowane jako wyjścia — ryzyko przy kolejnym uruchomieniu.

9. Odniesienia literaturowe

1. Dokumentacja Raspberry Pi — GPIO usage, hardware PWM, electrical specifications.
<https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>
2. ST Microelectronics — L298 Dual Full-Bridge Driver. Datasheet.
Specyfikacja prądów wyjściowych, napięć logicznych, tabela prawdy mostka H.
3. RPi.GPIO Python Library Documentation.
`GPIO.setmode()`, `GPIO.PWM`, `ChangeDutyCycle()` — interfejs programowy GPIO.
4. Kaehler A., Bradski G. — Learning OpenCV 3, O'Reilly 2016.
Kontekst całego systemu wizyjnego generującego sygnały sterujące dla `driver.py`.
5. Ogata K. — Modern Control Engineering, Pearson 2010.
Rozdział 2: Silniki DC jako elementy wykonawcze układów sterowania — model matematyczny, moment obrotowy, charakterystyki mechaniczne.